Boston Housing Price Prediction Using a PyTorch Neural Network

This notebook implements Boston housing price prediction with a deep neural network in PyTorch, using CUDA when available.

Workflow and Metrics

1. Load the Boston housing dataset
2. Display the dataset with column names
3. Clean and standardize the data
4. Train a PyTorch regression model
5. Evaluate with MAE, MSE, RMSE, and R2
6. Compare against a mean baseline

In [1]:
```
import torch

print("Torch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("Torch CUDA version:", torch.version.cuda)
print("GPU count:", torch.cuda.device_count())
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))
```

```
Torch version: 2.5.1+cu121
CUDA available: True
Torch CUDA version: 12.1
GPU count: 1
GPU name: NVIDIA GeForce RTX 3050 Laptop GPU
```

In [2]:
```
import random

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import fetch_openml
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, TensorDataset


def set_seed(seed: int = 42) -> None:
    random.seed(seed)
    np.random.seed(seed)
```

```
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)


set_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
if device.type == "cuda":
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("CUDA is not available in this environment.")
    print("Install CUDA-enabled PyTorch and verify NVIDIA driver/GPU visibility.")
    print("Recommended install command:")
    print("pip install --upgrade torch torchvision torchaudio --index-url https://d
```

```
Using device: cuda
GPU: NVIDIA GeForce RTX 3050 Laptop GPU
```

In [3]:

```
# Load Boston housing dataset
boston = fetch_openml(name="boston", version=1, as_frame=True)
data = boston.frame.copy()
data = data.rename(columns={"MEDV": "MEDV"})

feature_names = list(data.columns[:-1])
train_df = data.copy()
print("Dataset preview with column names:")
print(train_df.head())
print("\nMissing values:")
print(train_df.isnull().sum())

x = data.iloc[:, :-1].astype(np.float32).values
y = data.iloc[:, -1].astype(np.float32).values

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_sta

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

baseline_pred = np.full_like(y_test, y_train.mean(), dtype=np.float32)
baseline_mae = mean_absolute_error(y_test, baseline_pred)
baseline_mse = mean_squared_error(y_test, baseline_pred)
baseline_rmse = np.sqrt(baseline_mse)
baseline_r2 = r2_score(y_test, baseline_pred)

print("\nTrain shape:", x_train.shape, "Test shape:", x_test.shape)
print("Baseline MAE:", round(baseline_mae, 4))
print("Baseline MSE:", round(baseline_mse, 4))
print("Baseline RMSE:", round(baseline_rmse, 4))
print("Baseline R2:", round(baseline_r2, 4))
```

```
Dataset preview with column names:
      CRIM    ZN  INDUS CHAS    NOX     RM   AGE     DIS RAD    TAX  PTRATIO  \
0  0.00632  18.0   2.31    0  0.538  6.575  65.2  4.0900   1  296.0     15.3
1  0.02731   0.0   7.07    0  0.469  6.421  78.9  4.9671   2  242.0     17.8
2  0.02729   0.0   7.07    0  0.469  7.185  61.1  4.9671   2  242.0     17.8
3  0.03237   0.0   2.18    0  0.458  6.998  45.8  6.0622   3  222.0     18.7
4  0.06905   0.0   2.18    0  0.458  7.147  54.2  6.0622   3  222.0     18.7

        B  LSTAT  MEDV
0  396.90   4.98  24.0
1  396.90   9.14  21.6
2  392.83   4.03  34.7
3  394.63   2.94  33.4
4  396.90   5.33  36.2

Missing values:
CRIM       0
ZN         0
INDUS      0
CHAS       0
NOX        0
RM         0
AGE        0
DIS        0
RAD        0
TAX        0
PTRATIO    0
B          0
LSTAT      0
MEDV       0
dtype: int64

Train shape: (404, 13) Test shape: (102, 13)
Baseline MAE: 6.2558
Baseline MSE: 75.0454
Baseline RMSE: 8.6629
Baseline R2: -0.0233
```

In [4]:

```python
class BostonRegressor(nn.Module):
    def __init__(self, input_dim: int):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Linear(64, 1),
        )

    def forward(self, x):
        return self.network(x).squeeze(1)


model = BostonRegressor(x_train.shape[1]).to(device)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
x_train_tensor = torch.tensor(x_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
train_loader = DataLoader(TensorDataset(x_train_tensor, y_train_tensor), batch_size

history = {"train_loss": [], "val_loss": []}
best_val_loss = float("inf")
early_stop_counter = 0
patience = 15

for epoch in range(1, 201):
    model.train()
    running_loss = 0.0
    for batch_x, batch_y in train_loader:
        batch_x = batch_x.to(device)
        batch_y = batch_y.to(device)
        optimizer.zero_grad()
        outputs = model(batch_x)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * batch_x.size(0)

    train_loss = running_loss / len(train_loader.dataset)
    model.eval()
    with torch.no_grad():
        val_outputs = model(torch.tensor(x_test, dtype=torch.float32).to(device))
        val_loss = criterion(val_outputs, torch.tensor(y_test, dtype=torch.float32)

    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    print(f"Epoch {epoch}: train_loss={train_loss:.4f}, val_loss={val_loss:.4f}")

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_state = model.state_dict()
        early_stop_counter = 0
    else:
        early_stop_counter += 1
        if early_stop_counter >= patience:
            print("Early stopping triggered")
            break

model.load_state_dict(best_state)
print("Best validation loss:", round(best_val_loss, 4))
```

```
Epoch 1: train_loss=595.3443, val_loss=511.9334
Epoch 2: train_loss=563.3442, val_loss=473.5323
Epoch 3: train_loss=507.4360, val_loss=407.3884
Epoch 4: train_loss=414.3962, val_loss=306.0157
Epoch 5: train_loss=291.5235, val_loss=182.4755
Epoch 6: train_loss=164.6936, val_loss=85.9483
Epoch 7: train_loss=84.6033, val_loss=49.3021
Epoch 8: train_loss=56.9749, val_loss=37.2872
Epoch 9: train_loss=41.4896, val_loss=30.9141
Epoch 10: train_loss=32.8713, val_loss=27.6985
Epoch 11: train_loss=27.7405, val_loss=25.8164
Epoch 12: train_loss=25.1068, val_loss=24.4879
Epoch 13: train_loss=23.3763, val_loss=23.2390
Epoch 14: train_loss=22.2079, val_loss=22.3521
Epoch 15: train_loss=21.0685, val_loss=21.5886
Epoch 16: train_loss=20.2831, val_loss=20.8591
Epoch 17: train_loss=19.5663, val_loss=20.1061
Epoch 18: train_loss=18.9071, val_loss=19.6171
Epoch 19: train_loss=18.4591, val_loss=19.1094
Epoch 20: train_loss=17.7490, val_loss=18.5687
Epoch 21: train_loss=17.3927, val_loss=18.1304
Epoch 22: train_loss=16.8946, val_loss=17.7000
Epoch 23: train_loss=16.6097, val_loss=17.4993
Epoch 24: train_loss=16.2116, val_loss=16.9693
Epoch 25: train_loss=15.7121, val_loss=16.7689
Epoch 26: train_loss=15.3921, val_loss=16.5351
Epoch 27: train_loss=15.1103, val_loss=16.1507
Epoch 28: train_loss=14.9212, val_loss=15.9709
Epoch 29: train_loss=14.5305, val_loss=15.6239
Epoch 30: train_loss=14.3204, val_loss=15.4956
Epoch 31: train_loss=14.2971, val_loss=15.2026
Epoch 32: train_loss=13.7888, val_loss=15.0463
Epoch 33: train_loss=13.6347, val_loss=14.8825
Epoch 34: train_loss=13.4110, val_loss=14.7825
Epoch 35: train_loss=13.3126, val_loss=14.5977
Epoch 36: train_loss=13.1218, val_loss=14.5344
Epoch 37: train_loss=12.9107, val_loss=14.2835
Epoch 38: train_loss=12.7428, val_loss=14.1144
Epoch 39: train_loss=12.6150, val_loss=14.1337
Epoch 40: train_loss=12.4369, val_loss=13.9888
Epoch 41: train_loss=12.2909, val_loss=13.8788
Epoch 42: train_loss=12.2279, val_loss=13.7322
Epoch 43: train_loss=12.1395, val_loss=13.6446
Epoch 44: train_loss=12.0136, val_loss=13.5418
Epoch 45: train_loss=11.9029, val_loss=13.4182
Epoch 46: train_loss=11.9523, val_loss=13.3135
Epoch 47: train_loss=11.6205, val_loss=13.3165
Epoch 48: train_loss=11.5076, val_loss=13.2055
Epoch 49: train_loss=11.5814, val_loss=13.2353
Epoch 50: train_loss=11.3644, val_loss=13.1126
Epoch 51: train_loss=11.2319, val_loss=13.0671
Epoch 52: train_loss=11.1485, val_loss=12.9707
Epoch 53: train_loss=11.1679, val_loss=12.8521
Epoch 54: train_loss=10.9377, val_loss=12.8395
Epoch 55: train_loss=10.9040, val_loss=12.7578
Epoch 56: train_loss=10.8419, val_loss=12.6874
```

```
Epoch 57: train_loss=10.8463, val_loss=12.6472
Epoch 58: train_loss=10.6921, val_loss=12.6195
Epoch 59: train_loss=10.5469, val_loss=12.5299
Epoch 60: train_loss=10.4878, val_loss=12.4091
Epoch 61: train_loss=10.3886, val_loss=12.3922
Epoch 62: train_loss=10.3098, val_loss=12.3146
Epoch 63: train_loss=10.3337, val_loss=12.3494
Epoch 64: train_loss=10.2473, val_loss=12.2958
Epoch 65: train_loss=10.3768, val_loss=12.2598
Epoch 66: train_loss=10.1162, val_loss=12.3037
Epoch 67: train_loss=10.1012, val_loss=12.1041
Epoch 68: train_loss=9.8188, val_loss=12.1724
Epoch 69: train_loss=9.8755, val_loss=11.9639
Epoch 70: train_loss=9.8760, val_loss=11.9743
Epoch 71: train_loss=9.7948, val_loss=12.1324
Epoch 72: train_loss=9.5631, val_loss=11.9804
Epoch 73: train_loss=9.5518, val_loss=11.8502
Epoch 74: train_loss=9.5522, val_loss=11.9126
Epoch 75: train_loss=9.5204, val_loss=11.7793
Epoch 76: train_loss=9.3625, val_loss=11.8249
Epoch 77: train_loss=9.3343, val_loss=11.7382
Epoch 78: train_loss=9.2236, val_loss=11.7601
Epoch 79: train_loss=9.1639, val_loss=11.7636
Epoch 80: train_loss=9.0664, val_loss=11.6424
Epoch 81: train_loss=9.0507, val_loss=11.6480
Epoch 82: train_loss=8.9755, val_loss=11.5545
Epoch 83: train_loss=8.8779, val_loss=11.7104
Epoch 84: train_loss=8.8919, val_loss=11.4766
Epoch 85: train_loss=8.8087, val_loss=11.5940
Epoch 86: train_loss=8.7524, val_loss=11.5303
Epoch 87: train_loss=8.6751, val_loss=11.5510
Epoch 88: train_loss=8.7049, val_loss=11.4976
Epoch 89: train_loss=8.7169, val_loss=11.4632
Epoch 90: train_loss=8.5612, val_loss=11.4241
Epoch 91: train_loss=8.4799, val_loss=11.4472
Epoch 92: train_loss=8.4776, val_loss=11.3681
Epoch 93: train_loss=8.4088, val_loss=11.3804
Epoch 94: train_loss=8.3237, val_loss=11.3158
Epoch 95: train_loss=8.3564, val_loss=11.3543
Epoch 96: train_loss=8.3029, val_loss=11.2557
Epoch 97: train_loss=8.2129, val_loss=11.2255
Epoch 98: train_loss=8.1098, val_loss=11.3101
Epoch 99: train_loss=8.0745, val_loss=11.3960
Epoch 100: train_loss=8.0818, val_loss=11.2277
Epoch 101: train_loss=7.9918, val_loss=11.2764
Epoch 102: train_loss=7.9254, val_loss=11.2885
Epoch 103: train_loss=8.0043, val_loss=11.2989
Epoch 104: train_loss=7.9598, val_loss=11.2440
Epoch 105: train_loss=7.8088, val_loss=11.2036
Epoch 106: train_loss=7.8493, val_loss=11.1987
Epoch 107: train_loss=7.6683, val_loss=11.2231
Epoch 108: train_loss=7.6969, val_loss=11.2219
Epoch 109: train_loss=7.6899, val_loss=11.1682
Epoch 110: train_loss=7.6251, val_loss=11.0492
Epoch 111: train_loss=7.5679, val_loss=11.1241
Epoch 112: train_loss=7.5092, val_loss=11.0479
```

```
Epoch 113: train_loss=7.3885, val_loss=11.2316
Epoch 114: train_loss=7.4295, val_loss=11.0842
Epoch 115: train_loss=7.3925, val_loss=11.0798
Epoch 116: train_loss=7.3203, val_loss=11.1248
Epoch 117: train_loss=7.2767, val_loss=11.0445
Epoch 118: train_loss=7.2202, val_loss=11.0560
Epoch 119: train_loss=7.1484, val_loss=11.0710
Epoch 120: train_loss=7.2939, val_loss=11.1154
Epoch 121: train_loss=7.2440, val_loss=11.0334
Epoch 122: train_loss=7.0752, val_loss=11.1101
Epoch 123: train_loss=7.1769, val_loss=11.1757
Epoch 124: train_loss=6.9966, val_loss=11.1343
Epoch 125: train_loss=6.9077, val_loss=11.0246
Epoch 126: train_loss=6.8954, val_loss=11.0173
Epoch 127: train_loss=6.7861, val_loss=10.9932
Epoch 128: train_loss=6.7676, val_loss=11.0872
Epoch 129: train_loss=6.7825, val_loss=11.0924
Epoch 130: train_loss=6.7601, val_loss=10.9498
Epoch 131: train_loss=6.6864, val_loss=10.9946
Epoch 132: train_loss=6.7036, val_loss=10.9130
Epoch 133: train_loss=6.6257, val_loss=11.0538
Epoch 134: train_loss=6.6159, val_loss=10.9345
Epoch 135: train_loss=6.5095, val_loss=10.9818
Epoch 136: train_loss=6.4697, val_loss=11.0326
Epoch 137: train_loss=6.4862, val_loss=10.9112
Epoch 138: train_loss=6.3774, val_loss=11.1480
Epoch 139: train_loss=6.3846, val_loss=10.9833
Epoch 140: train_loss=6.2381, val_loss=10.9157
Epoch 141: train_loss=6.3136, val_loss=10.9559
Epoch 142: train_loss=6.2833, val_loss=10.9672
Epoch 143: train_loss=6.2178, val_loss=10.9531
Epoch 144: train_loss=6.1982, val_loss=10.9454
Epoch 145: train_loss=6.0582, val_loss=10.9567
Epoch 146: train_loss=6.0782, val_loss=10.9872
Epoch 147: train_loss=6.0035, val_loss=10.8868
Epoch 148: train_loss=5.9925, val_loss=10.8625
Epoch 149: train_loss=5.9206, val_loss=10.8865
Epoch 150: train_loss=5.9636, val_loss=10.9773
Epoch 151: train_loss=5.8572, val_loss=10.8337
Epoch 152: train_loss=5.8699, val_loss=10.8583
Epoch 153: train_loss=5.8932, val_loss=10.9623
Epoch 154: train_loss=5.8137, val_loss=10.8468
Epoch 155: train_loss=5.9047, val_loss=10.8530
Epoch 156: train_loss=5.7071, val_loss=10.8716
Epoch 157: train_loss=5.7519, val_loss=10.9215
Epoch 158: train_loss=5.6312, val_loss=10.7926
Epoch 159: train_loss=5.5730, val_loss=10.7897
Epoch 160: train_loss=5.5670, val_loss=10.8598
Epoch 161: train_loss=5.5216, val_loss=10.8197
Epoch 162: train_loss=5.6157, val_loss=11.0135
Epoch 163: train_loss=5.5214, val_loss=10.7865
Epoch 164: train_loss=5.4599, val_loss=10.7945
Epoch 165: train_loss=5.4534, val_loss=10.7968
Epoch 166: train_loss=5.3704, val_loss=10.7473
Epoch 167: train_loss=5.3477, val_loss=10.8413
Epoch 168: train_loss=5.3357, val_loss=10.6303
```

```
Epoch 169: train_loss=5.2682, val_loss=10.7602
Epoch 170: train_loss=5.3682, val_loss=10.6583
Epoch 171: train_loss=5.2023, val_loss=10.6757
Epoch 172: train_loss=5.1959, val_loss=10.6806
Epoch 173: train_loss=5.2024, val_loss=10.6268
Epoch 174: train_loss=5.1546, val_loss=10.7192
Epoch 175: train_loss=5.1184, val_loss=10.7161
Epoch 176: train_loss=5.1602, val_loss=10.5816
Epoch 177: train_loss=5.1161, val_loss=10.6191
Epoch 178: train_loss=5.0460, val_loss=10.6684
Epoch 179: train_loss=4.9700, val_loss=10.5963
Epoch 180: train_loss=4.9575, val_loss=10.5609
Epoch 181: train_loss=4.9122, val_loss=10.5867
Epoch 182: train_loss=4.8685, val_loss=10.7110
Epoch 183: train_loss=4.9688, val_loss=10.5205
Epoch 184: train_loss=4.8684, val_loss=10.5521
Epoch 185: train_loss=4.8061, val_loss=10.5560
Epoch 186: train_loss=4.8286, val_loss=10.5216
Epoch 187: train_loss=4.8477, val_loss=10.7010
Epoch 188: train_loss=4.8965, val_loss=10.5669
Epoch 189: train_loss=4.7191, val_loss=10.4767
Epoch 190: train_loss=4.7566, val_loss=10.5711
Epoch 191: train_loss=4.6915, val_loss=10.6055
Epoch 192: train_loss=4.6956, val_loss=10.5757
Epoch 193: train_loss=4.6752, val_loss=10.3566
Epoch 194: train_loss=4.7515, val_loss=10.5576
Epoch 195: train_loss=4.6599, val_loss=10.4398
Epoch 196: train_loss=4.5826, val_loss=10.4934
Epoch 197: train_loss=4.5379, val_loss=10.5615
Epoch 198: train_loss=4.4860, val_loss=10.4279
Epoch 199: train_loss=4.5346, val_loss=10.5020
Epoch 200: train_loss=4.4636, val_loss=10.3739
Best validation loss: 10.3566
```

In [5]:

```
model.eval()
with torch.no_grad():
    y_pred = model(torch.tensor(x_test, dtype=torch.float32).to(device)).cpu().nump

model_mae = mean_absolute_error(y_test, y_pred)
model_mse = mean_squared_error(y_test, y_pred)
model_rmse = np.sqrt(model_mse)
model_r2 = r2_score(y_test, y_pred)

print("Model MAE:", round(model_mae, 4))
print("Model MSE:", round(model_mse, 4))
print("Model RMSE:", round(model_rmse, 4))
print("Model R2:", round(model_r2, 4))
print("\nComparison (Baseline vs Model):")
print(f"MAE  : {baseline_mae:.4f} -> {model_mae:.4f}")
print(f"MSE  : {baseline_mse:.4f} -> {model_mse:.4f}")
print(f"RMSE : {baseline_rmse:.4f} -> {model_rmse:.4f}")
print(f"R2   : {baseline_r2:.4f} -> {model_r2:.4f}")

plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r--")
```

```
plt.xlabel("Actual price")
plt.ylabel("Predicted price")
plt.title("Actual vs Predicted Boston Housing Prices")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Model MAE: 2.1703
Model MSE: 10.3739
Model RMSE: 3.2209
Model R2: 0.8585

Comparison (Baseline vs Model):
MAE  : 6.2558 -> 2.1703
MSE  : 75.0454 -> 10.3739
RMSE : 8.6629 -> 3.2209
R2   : -0.0233 -> 0.8585
```